

UNIVERSITY OF MADRAS
M.Sc. DEGREE PROGRAMME IN COMPUTER SCIENCE
SYLLABUS WITH EFFECT FROM 2023-2024

Title of the Paper	Principles of Compiler Design		
Elective – II - Theory	I Year & I Semester	Credit:3	436E1D

Objectives:

- To learn the various phases of compiler.
- To learn the various parsing techniques.
- To understand intermediate code generation and run-time environment.
- To learn to implement the front-end of the compiler.
- To learn to implement code generators.

Outcomes:

1.	Understand the different phases of the compiler.	K1, K2
2.	Design a lexical analyzer for a sample language.	K2, K3
3.	Apply different parsing algorithms to develop the parsers for a given grammar.	K3, K4
4.	Design and implement a scanner and a parser using LEX and YACC tools	K4, K5
5.	Learn to implement code optimization techniques and a simple code generator.	K5, K6
K1-Remember;K2-Understand;K3-Apply;K4-Analyze;K5-Evaluate; K6-Create		

UNIT I: INTRODUCTION TO COMPILERS: Structure of a compiler – Lexical Analysis – Role of Lexical Analyzer – Input Buffering – Specification of Tokens – Recognition of Tokens – Lex – Finite Automata – Regular Expressions to Automata – Minimizing DFA.

UNIT II: SYNTAX ANALYSIS: Role of Parser – Grammars – Error Handling – Context free grammars – Writing a grammar – Top Down Parsing - General Strategies Recursive Descent Parser Predictive Parser-LL(1) Parser-Shift Reduce Parser-LR Parser- LR (0)Item Construction of SLR Parsing Table - Introduction to LALR Parser - Error Handling and Recovery in Syntax Analyzer-YACC.

UNIT III: INTERMEDIATE CODE GENERATION: Syntax Directed Definitions, Evaluation Orders for Syntax Directed Definitions, Intermediate Languages: Syntax Tree, Three Address Code, Types and Declarations, Translation of Expressions, Type Checking.

UNIT IV: RUN-TIME ENVIRONMENT AND CODE GENERATION: Storage Organization, Stack Allocation Space, Access to Non-local Data on the Stack, Heap Management - Issues in Code Generation - Design of a simple Code Generator.

UNIT V: CODE OPTIMIZATION: Principal Sources of Optimization – Peep-hole optimization - DAG- Optimization of Basic Blocks- Global Data Flow Analysis - Efficient Data Flow Algorithm.

UNIVERSITY OF MADRAS
M.Sc. DEGREE PROGRAMME IN COMPUTER SCIENCE
SYLLABUS WITH EFFECT FROM 2023-2024

Recommended Texts:

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, Compilers: Principles, Techniques and Tools; Pearson Education; Second Edition; 2013

Reference Books:

1. Randy Allen, Ken Kennedy, Optimizing Compilers for Modern Architectures: A Dependence based Approach, Morgan Kaufmann Publishers, 2002.
2. Steven S. Muchnick, Advanced Compiler Design and Implementation, Morgan Kaufmann Publishers - Elsevier Science, India, Indian Reprint 2003.
3. Keith D Cooper and Linda Torczon, Engineering a Compiler, Morgan Kaufmann Publishers Elsevier Science, 2004.
4. V. Raghavan, Principles of Compiler Design, Tata McGraw Hill Education Publishers, 2010.
5. Allen I. Holub, Compiler Design in C, Prentice-Hall Software Series, 1993.

Web References:

1. https://www.youtube.com/watch?v=k4QXWFZZq1E&list=PLENQMW_c1dimxH Uu6KjuBC2rOlAaoLozF

Mapping with Programme Outcomes:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10
CO1	M	S	S	L	S	M	L	M	L	L
CO2	S	M	S	S	S	M	M	L	M	L
CO3	S	S	L	M	S	S	L	M	L	M
CO4	M	L	S	M	M	L	S	L	M	S
CO5	S	S	M	S	L	M	M	L	M	L

S-Strong M-Medium L-Low